Dev2 구현 계획 — 비전공자도 이해할 수 있는 설명

> 원본 문서: `2026-05-22-dev2-implementation-plan.md` 이 문서는 코딩을 모르는 사람도 "Dev2가 뭘 만들고 있는지" 이해할 수 있도록 풀어 쓴 설명서입니다.

한 줄 요약

Dev2가 만드는 것: 신입 개발자가 GitHub에 코드를 올리면, AI가 자동으로 "이 코드에 문제가 있는지" 검사하고 결과를 정리해주는 시스템

큰 그림 — Dev2는 팀 전체 시스템의 어디에 있나?

```
신입 개발자가 코드 올림 (GitHub PR)
        ↓
   Dev1 (교통정리 담당)
        ↓
   ┌──────────────────┐
   │   Dev2 ← 여기    │  코드 품질 + 보안 문제 찾기
   └──────────────────┘
        ↓
   Dev3 (테스트 충분한지 검사)
        ↓
   Dev4 (최종 리뷰 코멘트 작성 + Slack 알림)
        ↓
   Dev5 (학습 커리큘럼 추천)
```

Dev2는 "문제 탐정" 역할이에요.
코드를 받아서 문제점을 찾아내고 → 결과를 정리해서 다음 단계(Dev4)에 넘겨줍니다.

Dev2가 하는 일 전체 흐름

코드가 들어오면 Dev2는 두 단계로 처리해요.

파이프라인 1 — "코드 검사" (POST /scan)

```
코드 입력
   ↓
① 어떤 도구로 검사할지 결정  (tool_router_agent)
   ↓
② 실제로 검사 도구 실행      (run_tools)
   ↓
③ 과거 비슷한 문제 찾기      (vector_search)
   ↓
④ AI가 심각도 최종 판단      (llm_judge)
```

```
   ↓
⑤ 다음 행동 추천              (action_decider_agent)
   ↓
⑥ 결과 저장                    (store)
```

파이프라인 2 — "결과 정리 + 알림" (POST /aggregate)

```
검사 완료
   ↓
① 이슈 합산 + 학습 정보 추가 (aggregate)
   ↓
② AI가 개발자 수준에 맞는 코멘트 작성 (comment_refine)
   ↓
③ 심각한 문제 있으면 → Slack 알림 (slack_notify)
   ↓
④ GitHub PR에 코멘트 등록     (github_comment)
   ↓
⑤ 학습 담당(M4)에게 결과 전달
```

Task별 설명

Task 1: 개발 환경 세팅 (Docker)

비유: 요리를 시작하기 전에 주방을 준비하는 것

Docker란?
프로그램을 실행하는 데 필요한 모든 환경을 박스(컨테이너)에 담아두는 기술이에요.
"내 컴퓨터에서는 됐는데 다른 컴퓨터에선 안 돼요" 문제를 없애줍니다.

이 Task에서 만드는 것:

- `Dockerfile` — 프로그램을 담을 박스의 설계도
- `docker-compose.yml` — 여러 박스(프로그램)를 함께 실행하는 방법
- `requirements.txt` — 필요한 외부 라이브러리 목록

완료 기준: 서버가 정상적으로 켜지는 것 확인

Task 2: 데이터 구조 설계 (Schema + DB)

비유: 파일 보관함을 만들기 전에 어떤 서랍을 몇 개 만들지 결정하는 것

Schema(스키마)란?
데이터를 어떤 형태로 저장할지 정의한 설계도예요.
예: "이슈 하나는 심각도, 파일명, 줄 번호, 설명을 가져야 한다"

DB(데이터베이스)란?
데이터를 체계적으로 보관하는 창고예요.
엑셀의 방대한 버전이라고 생각하면 됩니다.

이 Task에서 만드는 것:

- `scan_executions` 테이블 — "언제 누가 어떤 PR을 스캔했나" 기록
- `scan_issues` 테이블 — "어떤 문제가 몇 번째 줄에서 발견됐나" 기록
- `scan_summaries` 테이블 — "이번 스캔에서 Critical/Major/Minor가 몇 개였나" 요약

완료 기준: 테이블이 실제로 생성되고, 데이터를 넣고 뺄 수 있음

Task 3: 검사 도구들 (CLI Tools)

비유: 탐정이 사용하는 다양한 조사 장비들을 준비하는 것

실제로 코드를 분석하는 자동화 도구들이에요.

도구	역할	비유
Pylint	Python 코드 규칙 위반 찾기	맞춤법 검사기
Bandit	Python 보안 취약점 찾기	금고 잠금 검사기
Radon	코드 복잡도 측정	설명서 가독성 검사기
ESLint	JavaScript 코드 규칙 위반 찾기	JS용 맞춤법 검사기
Semgrep	전 언어 공통 보안 패턴 찾기	만능 보안 스캐너
pip-audit	Python 패키지 취약점 찾기	재료 유통기한 검사기

언어별 자동 선택: `.py` 파일이면 Python 도구, `.js` 파일이면 JS 도구를 자동으로 고릅니다.

Task 4-a: 도구 실행 노드 (run_tools)

비유: 탐정들을 현장에 파견해서 각자 맡은 조사를 동시에 하게 하는 것

코드 파일 목록을 받아서 언어별 도구들을 **동시에** 실행합니다.
순서대로 하면 오래 걸리니까, 여러 도구를 동시에 돌려서 시간을 절약해요.

핵심: `tool_plan`이 있으면 선택된 도구만, 없으면 모든 도구 실행 (자동 fallback)

Task 4-b: 프롬프트 엔지니어링 (Prompts)

비유: AI에게 "어떻게 판단해줘"라는 지침서를 잘 작성하는 것

프롬프트(Prompt)란?
AI(Claude)에게 보내는 질문 + 지시사항이에요.
같은 정보를 줘도 질문을 어떻게 쓰느냐에 따라 AI의 답이 달라집니다.

Dev2에서 쓰는 기법:

- **Few-shot**: 예시를 3개 먼저 보여줌 ("이런 경우는 Critical, 이런 경우는 Minor")
- **Chain-of-Thought**: 단계별로 생각하게 함 ("1단계: 패턴 파악 → 2단계: 과거 사례 비교 → 3단계: 결론")
- **Structured Output**: JSON 형식으로만 답하게 강제함 (파싱 실패 방지)
- **RAG**: 과거 유사 사례를 찾아서 프롬프트에 끼워 넣음

Task 4-c: AI 심각도 판단 노드 (llm_judge)

비유: 선배 탐정이 증거를 보고 "이건 긴급, 저건 나중에"로 분류하는 것

도구가 찾은 문제들을 AI(Claude)가 다시 한번 검토해서 심각도를 최종 결정합니다.

모델 라우팅 (비용 절감 전략):

```
Critical/Major 문제 → 비싼 모델(claude-sonnet) 사용 (정확도 중요)
Minor 문제          → AI 판단 생략 (도구 결과 그대로 사용)
```

USE_MOCK_LLM=true: 개발 중에는 실제 AI를 안 부르고 가짜 응답을 씀 → 비용 0원

Task 4-d: 유사 이슈 검색 (vector_search)

비유: "이전에 비슷한 사건이 있었는지" 수사 기록을 뒤지는 것

Vector DB (pgVector)란?
텍스트의 "의미"를 숫자로 변환해서 비슷한 텍스트를 빠르게 찾는 기술이에요.
"SQL 인젝션 가능성"이라는 문장과 "SQL 파라미터 미처리"라는 문장이 비슷하다는 걸 알아냅니다.

왜 쓰나?
과거에 같은 유형의 문제가 있었다면 → AI가 더 정확한 판단을 내릴 수 있어요.

Task 4-e: LLM 평가 (Evaluation Harness)

비유: AI 탐정이 얼마나 정확한지 정기 테스트하는 것

LangSmith evaluate()란?
"정답이 있는 문제" 10개를 AI에게 풀게 하고 점수를 자동으로 측정하는 도구예요.

```
예: "SQL 파라미터 미처리" → 정답: Critical
    AI가 Critical이라고 답하면 1점, 아니면 0점
    → 정확도 87%라고 측정
```

이 수치가 있으면 "프롬프트를 바꿨더니 정확도가 87% → 93%로 올랐다"고 말할 수 있어요.

Task 4-f: 결과 저장 노드 (store)

비유: 조사 결과 보고서를 두 군데 파일링하는 것

검사 결과를 두 곳에 동시에 저장합니다.

저장소	목적	비유
PostgreSQL	실시간 조회용	현장 파일함 (빠른 접근)
BigQuery	분석/통계용	중앙 기록 보관소 (대량 분석)

PostgreSQL은 "방금 스캔한 PR 결과"를 빠르게 꺼내올 때, BigQuery는 "지난 3달간 SQL 인젝션이 몇 번 나왔나"를 집계할 때 씁니다.

Task 4-g: M4 학습 연결 필드 (concept_mapper)

비유: 문제 보고서에 "공부해야 할 개념" 태그를 붙이는 것

이슈마다 학습 담당(M4)이 이해할 수 있는 분류 정보를 자동으로 붙입니다.

```
발견된 이슈:
  rule_id: B608
  message: "SQL 문자열 직접 조합"

자동으로 붙이는 태그:
  concept: "SQL Injection"        ← 어떤 보안 개념인지
  mistake_type: "raw_query"       ← 어떤 유형의 실수인지
  confidence_score: 0.92          ← AI가 얼마나 확신하는지
  code_snippet: "query = sql + x" ← 문제 코드 조각
  learning_context:
    framework: "django"           ← 어떤 프레임워크를 쓰는지
    language: "python"
```

이 정보를 받아서 M4가 "SQL Injection 방지 방법" 학습자료를 추천해줄 수 있어요.

Task 4-h: Agentic 노드 2개 추가 (tool_router + action_decider)

비유: 수사 시작 전 계획 세우기 + 수사 끝나고 다음 행동 결정하기

tool_router_agent — "어떤 도구를 쓸지 먼저 결정"

매번 모든 도구를 다 쓰면 시간과 비용이 낭비됩니다.
PR 내용을 보고 "이번엔 이 도구들만 쓰면 충분하다"고 미리 결정합니다.

```
예시:
  CSS 파일만 바뀌었다 → stylelint만 실행 (다른 도구 스킵)
  auth.py + SQL 코드   → pylint + bandit + semgrep 실행 (보안 강화)
  package.json 변경    → npm_audit 추가 (패키지 취약점 검사)
  이전 PR에 SQL 인젝션 이력 있음 → 보안 도구 무조건 포함
```

action_decider_agent — "검사 끝나고 다음 행동 추천"

검사 결과를 보고 "다음에 어떻게 해야 하나"를 추천합니다.
중요한 것: **추천만 하고 직접 실행하지 않아요.** 실제 결정은 Dev4/M0가 합니다.

상황	추천 행동	Dev2가 하는 것
Critical 이슈 1개 이상	`early_escalation`	escalate 노드 진입 → 즉시 알림
Major 이슈 3개 이상	`request_rework`	store에 저장만 (M0/Dev4가 판단)
보안 중요 파일(auth, payment)에서 이슈	`needs_human_review`	store에 저장만
AI 분류 신뢰도 낮음	`needs_human_review`	store에 저장만
별 문제 없음	`store_only`	store에 저장만

Task 5: 파이프라인 1 전체 연결 (graph.py)

비유: 탐정들을 순서대로 배치해서 수사 흐름을 만드는 것

LangGraph란?
AI 에이전트들을 파이프라인처럼 연결해주는 프레임워크예요.
"A가 끝나면 B로 넘어가고, 조건에 따라 C 또는 D로 분기"하는 흐름을 코드로 표현합니다.

최종 파이프라인 구조 (7단계):

```
시작
 1. tool_router_agent  → "이번엔 이 도구들 써"
 2. run_tools          → 도구 실제 실행
 3. vector_search      → 과거 유사 사례 검색
 4. llm_judge          → AI가 심각도 최종 판단
 5. action_decider_agent → "다음엔 이렇게 해"
 6. (조건부) escalate  → Critical이면 즉시 알림
 7. store              → 결과 저장
끝
```

Task 6: 웹 API 만들기 (FastAPI)

비유: 시스템에 문을 달아서 외부에서 요청을 받을 수 있게 하는 것

FastAPI란?
파이썬으로 웹 API를 빠르게 만들 수 있는 도구예요.
"이 주소로 데이터를 보내면 이렇게 처리해줄게"를 정의합니다.

Dev2가 제공하는 API 목록:

API	역할	누가 부르나
`POST /scan`	코드 검사 시작	Dev1 (M0)

API	역할	누가 부르나
`POST /aggregate`	결과 취합 + 코멘트	Dev1 (M0)
`GET /insights/developer/{id}`	개발자별 이슈 이력 조회	M4 (참고용)
`GET /insights/team`	팀 전체 Top5 이슈	M4 (참고용)
`GET /insights/trends`	주간 이슈 추이	M4 (참고용)
`PATCH /issues/{id}/dismiss`	잘못된 탐지 표시	개발자 직접

202 Accepted란?
"접수했어요, 처리는 백그라운드에서 할게요"라는 응답이에요.
검사가 끝날 때까지 기다리지 않고 바로 응답을 줍니다.

Task 7: 자동 스케줄 작업 (Airflow DAG)

비유: 매일 밤 자동으로 돌아가는 청소 로봇처럼

Airflow란?
"매일 새벽 2시에 이 작업을 해줘", "매주 월요일 오전 9시에 저 작업을"처럼
반복 작업을 자동으로 예약하는 도구예요.

Dev2의 자동 작업들:

일별 (매일 새벽 2시):

- PostgreSQL에서 어제 스캔 결과를 꺼내서
- BigQuery 분석 창고로 옮기기

주별 (매주 월요일 오전 9시):

- 이번 주 이슈 패턴 통계 생성
- "오탐률(잘못된 탐지) 20% 넘는 규칙" 있으면 Slack 알림
- 보안 도구(Bandit, Semgrep) 최신 버전인지 확인 → 구버전이면 알림

Task 8 / 13: 전체 테스트 + E2E 확인

비유: 공장 가동 전에 전체 라인을 한 번 돌려보는 것

단위 테스트(Unit Test)란?
부품 하나하나가 제대로 작동하는지 검사하는 것이에요.

E2E(End-to-End) 테스트란?
처음부터 끝까지 전체 흐름이 정상 작동하는지 확인하는 것이에요.

테스트 현황:

```
test_concept_mapper.py      30개 통과 ✅
test_tool_router_agent.py   22개 통과 ✅
```

```
test_action_decider_agent.py 19개 통과 ✅
test_graph.py                5개 통과  ✅
```

Task 9~13: 파이프라인 2 (취합 + 코멘트 + 알림)

비유: 조사 보고서를 완성해서 관련자들에게 전달하는 것

파이프라인 1이 "문제 찾기"였다면, 파이프라인 2는 "결과를 사람이 읽기 좋게 만들어 전달하기"입니다.

Task	하는 일	비유
9 (aggregate)	M2 결과 합산 + DB 저장 + M4에 전달	최종 보고서 취합
10 (comment_refine)	AI가 개발자 수준에 맞는 설명 작성	고객 맞춤 보고서 작성
11 (slack_notify + github_comment)	Slack 알림 + GitHub 코멘트 등록	관련자에게 보고서 배송
12 (aggregate_graph)	파이프라인 2 전체 연결	배송 시스템 구축
13 (E2E 테스트)	파이프라인 2 전체 테스트	배송 테스트 운영

전체 Task 완료 체크리스트

Task	내용	상태
1	Docker + 환경 세팅	✅ 완료
2	데이터 구조 설계	✅ 완료
3	검사 도구들 구현	✅ 완료
4-a	run_tools 노드	✅ 완료
4-b	프롬프트 엔지니어링	✅ 완료
4-c	llm_judge 노드	✅ 완료
4-d	vector_search 노드	✅ 완료
4-e	LLM 평가 (Eval Harness)	✅ 완료
4-f	store 노드	✅ 완료
4-g	M4 개념 매핑 (concept_mapper)	✅ 완료
4-h	tool_router + action_decider	✅ 완료
5	파이프라인 1 전체 연결	✅ 완료
6	FastAPI 웹 API	✅ 완료
7	Airflow 자동 스케줄	✅ 완료
8	파이프라인 1 E2E 테스트	✅ 완료

Task	내용	상태
9	aggregate 노드	☐ 미완료
10	comment_refine 노드	☐ 미완료
11	github_comment + slack_notify	☐ 미완료
12	파이프라인 2 전체 연결	☐ 미완료
13	파이프라인 2 E2E 테스트	☐ 미완료

자주 나오는 용어 정리

용어	쉬운 설명
PR (Pull Request)	개발자가 "내가 만든 코드를 합쳐줘"라고 요청하는 것
Docker	프로그램 실행 환경을 박스에 담아두는 기술
API	프로그램끼리 소통하는 창구. "이 주소로 데이터 보내면 처리해줄게"
DB (데이터베이스)	데이터를 체계적으로 보관하는 창고
LangGraph	AI 에이전트들을 파이프라인으로 연결하는 도구
LLM	AI 언어 모델. 여기선 Claude(Anthropic) 사용
RAG	AI에게 질문할 때 관련 자료를 함께 넘겨서 더 정확한 답을 얻는 기법
pgVector	텍스트 의미로 유사한 것을 검색하는 DB 기술
Critical / Major / Minor	문제 심각도 분류. Critical=즉시 수정, Major=중요, Minor=권고
Airflow	반복 작업을 자동으로 예약하는 도구
테스트 (Test)	코드가 의도한 대로 작동하는지 자동으로 확인하는 코드
Mock	실제 외부 서비스 대신 가짜 응답을 쓰는 것. 개발 중 비용 절감용
PostgreSQL	데이터를 테이블 형식으로 저장하는 DB
BigQuery	구글의 대용량 데이터 분석 DB
FastAPI	Python으로 웹 API를 빠르게 만드는 도구
LangSmith	AI 에이전트 실행을 추적하고 품질을 측정하는 도구